bigt.ai

P2P 结算 — 钱包原生的人民币兑换

这是什么。 在 **Bigtangle L0** 上的加密货币与人民币法币（微信支付 / 支付宝 / 银行）之间的点对点兑换。卖方将代币托管到一个 **2-of-3 P2SH 地址**（卖方、买方、引擎）；买方在自己的支付应用里**直接**付款给卖方；卖方在自己的账单上确认收款后，托管随即释放。每一步不可逆操作都由钱包自己的 PQ 私钥签名，引擎只能**共同签名**一笔支出——它从不持有你的资金，也从不触碰法币。

在哪里。 钱包中的 **P2P** 页面（侧边栏 → 交易 → P2P）。以下所有截图都取自本地演示构建的该页面。

演示与线上。 截图来自演示构建：引擎运行在内存存储模式且不做链上检查，因此锁定/释放步骤使用合成的交易哈希。状态机、签名、只追加事件日志以及链上审计锚定都是真实的；只有 L0 广播被替换为桩实现。

卖方确认就是法币到账信号。 个人微信/支付宝/银行账户没有收款 API，因此不存在可供等待的、可机器验证的付款事件。卖方在**自己的应用里核对金额 + 备注 + 时间 + 流水号**——截图永远不算数——然后确认。设计细节见 `docs/p2p.md`。

流程一览

```
ACTIVE ──match──▶ MATCHED ──lock──▶ ESCROW_LOCKED ──instructions──▶ PAYMENT_PENDING
                                                                          │
   PAYMENT_PENDING ──proof──▶ PAYMENT_CLAIMED ──confirm──▶ PAYMENT_VERIFIED
                                                                          │
                                                                          ▼
                    PAYMENT_VERIFIED ──release──▶ ESCROW_RELEASED ──complete──▶ COMPLETED
```

#	步骤	操作方	链上 / 引擎
1	卖方保存人民币收款资料	卖方	资料存于服务端，绝不进入订单簿
2	卖方挂出签名的卖单	卖方	订单入库，尚未注资
3	买方撮合（收款地址）	买方	创建 `swapId`，推导托管地址

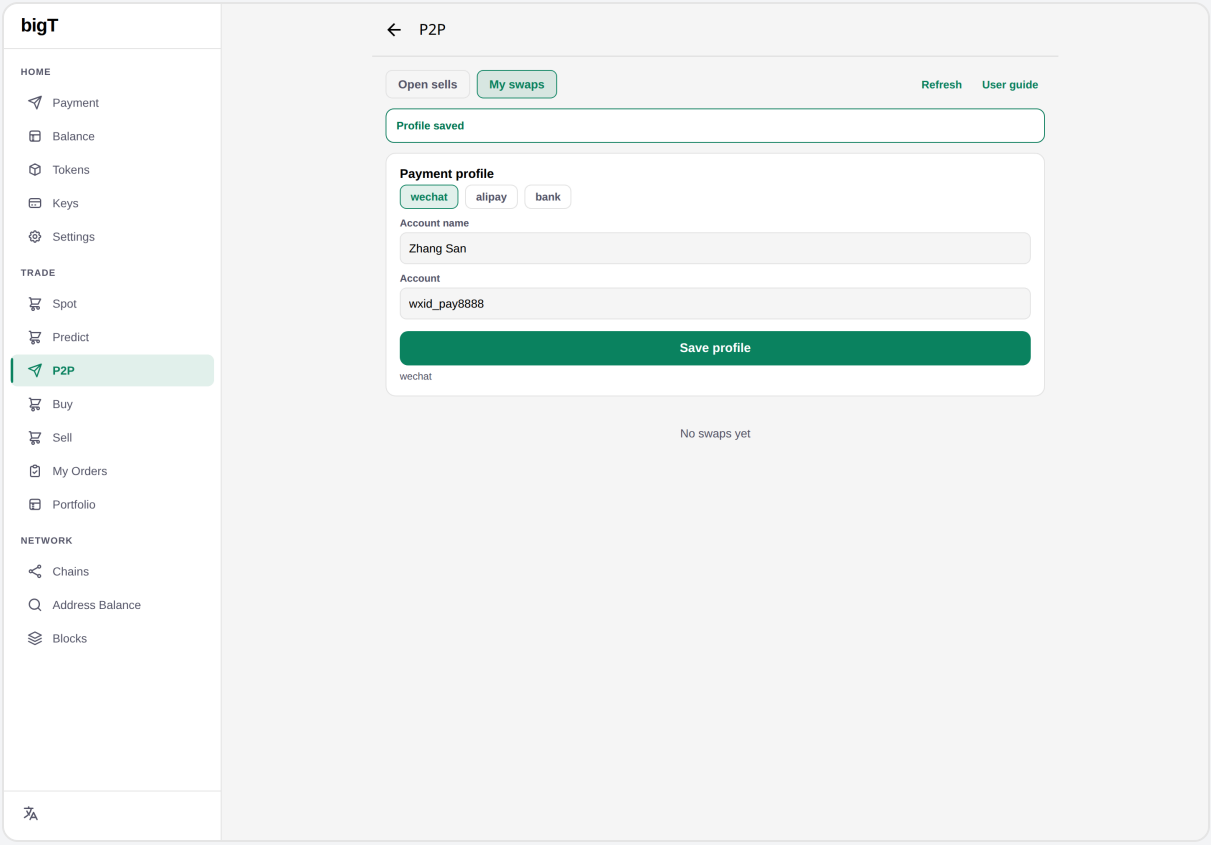
4	卖方为 2-of-3 托管注资，引擎证明锁定	卖方 + 引擎	L0 上 ESCROW_LOCKED
5	买方拉取付款说明	买方	PAYMENT_PENDING （+ 每笔交易的备注）
6	买方在自己的应用里付款，用流水号认领	买方	PAYMENT_CLAIMED
7	卖方核对自己的账单并确认	卖方	PAYMENT_VERIFIED
8	引擎释放代币，交易完成	引擎	ESCROW_RELEASED → COMPLETED

每次状态转换都会作为一条 social.p2p-swap 记录锚定到 L1-SOCIAL 链上，因此整个生命周期都可公开审计，同时不暴露任何一方的账户信息。

---

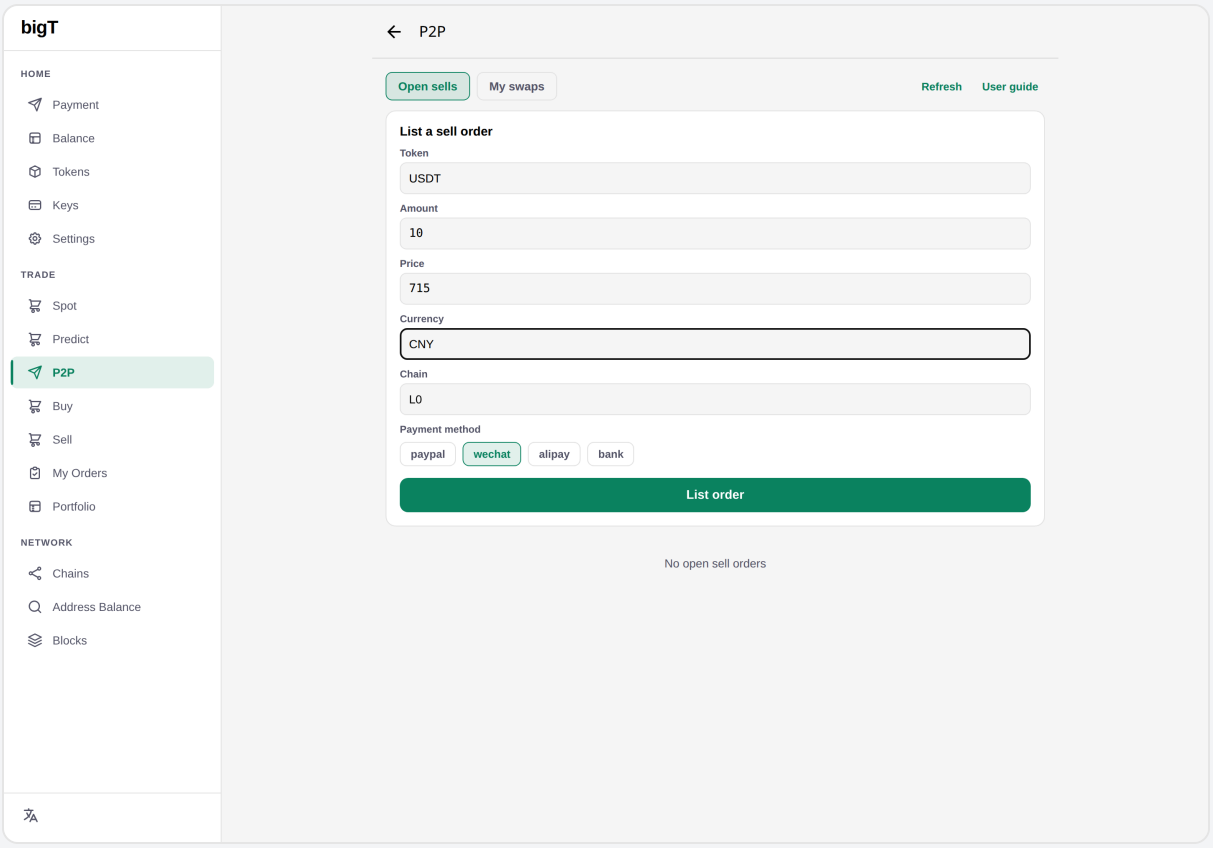
1. 卖方保存收款资料

在 **My swaps（我的交易）** 标签页，卖方选择一个渠道（微信、支付宝或银行），并填写收款名称与账号。资料存放在服务端、挂在卖方 DID 之下，且**不**属于公开订单簿——只有通过说明调用才会向已撮合的买方披露。



2. 卖方挂出卖单

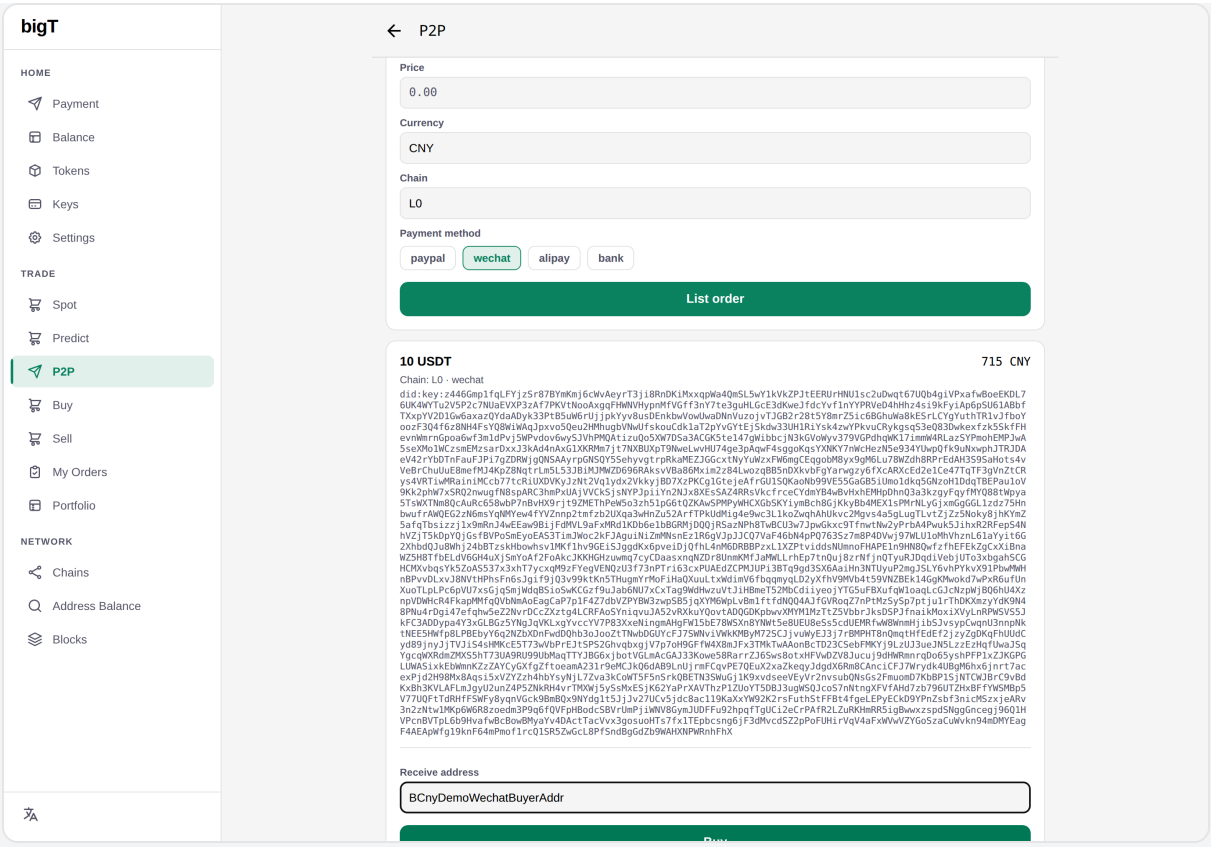
在 **`Open sells（卖单）`** 标签页填写订单：你给出的代币与数量、你想要的人民币价格、代币所在的链，以及**支付方式**（`wechat / alipay / bank`）。选择人民币渠道会将币种切换为 `CNY`。条款在挂单时即被锁定进订单，因此这一步用你的钱包密钥签名：



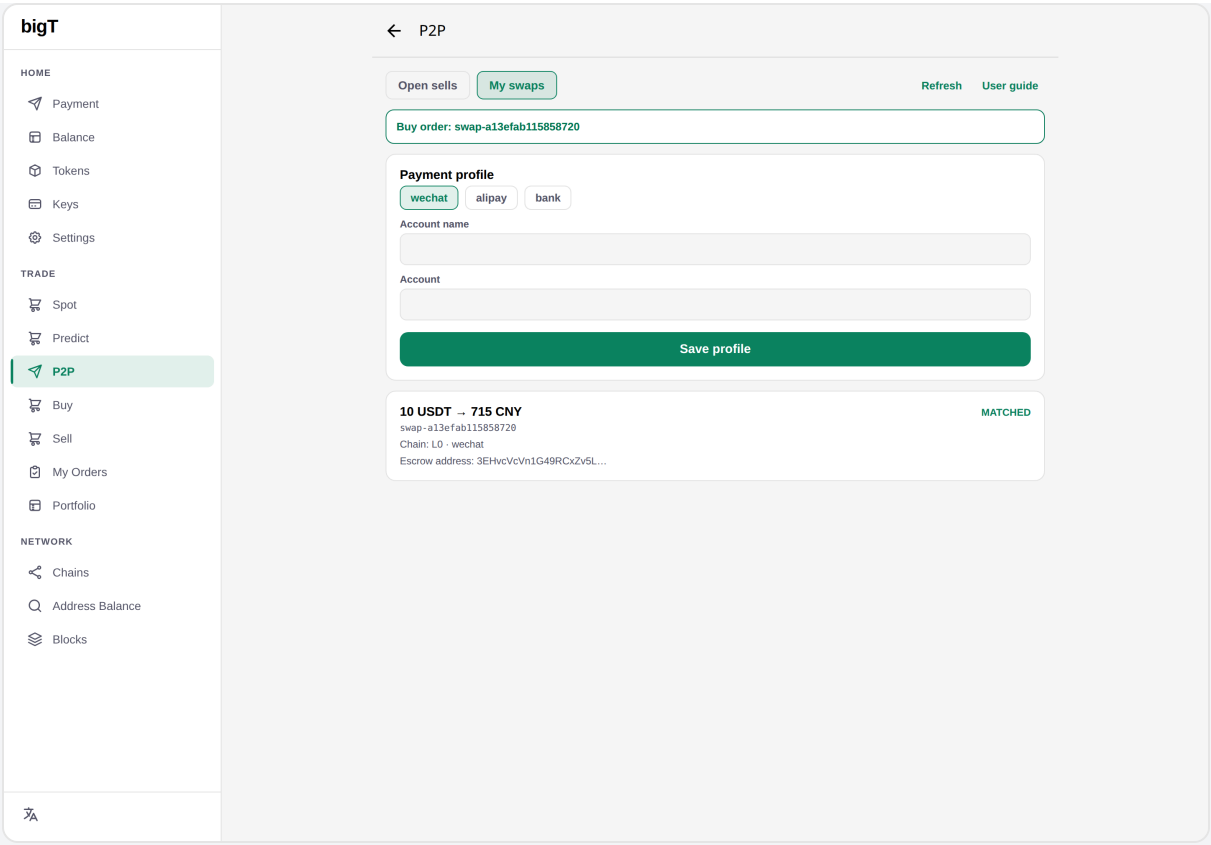
挂出后，订单在引擎中为 `ACTIVE`，并出现在公开订单簿中（订单不包含任何个人资料）。

3. 买方撮合

买方打开订单，提供用于接收释放代币的**收款地址**。撮合意味着买方承诺付款，因此这一步同样需要签名：



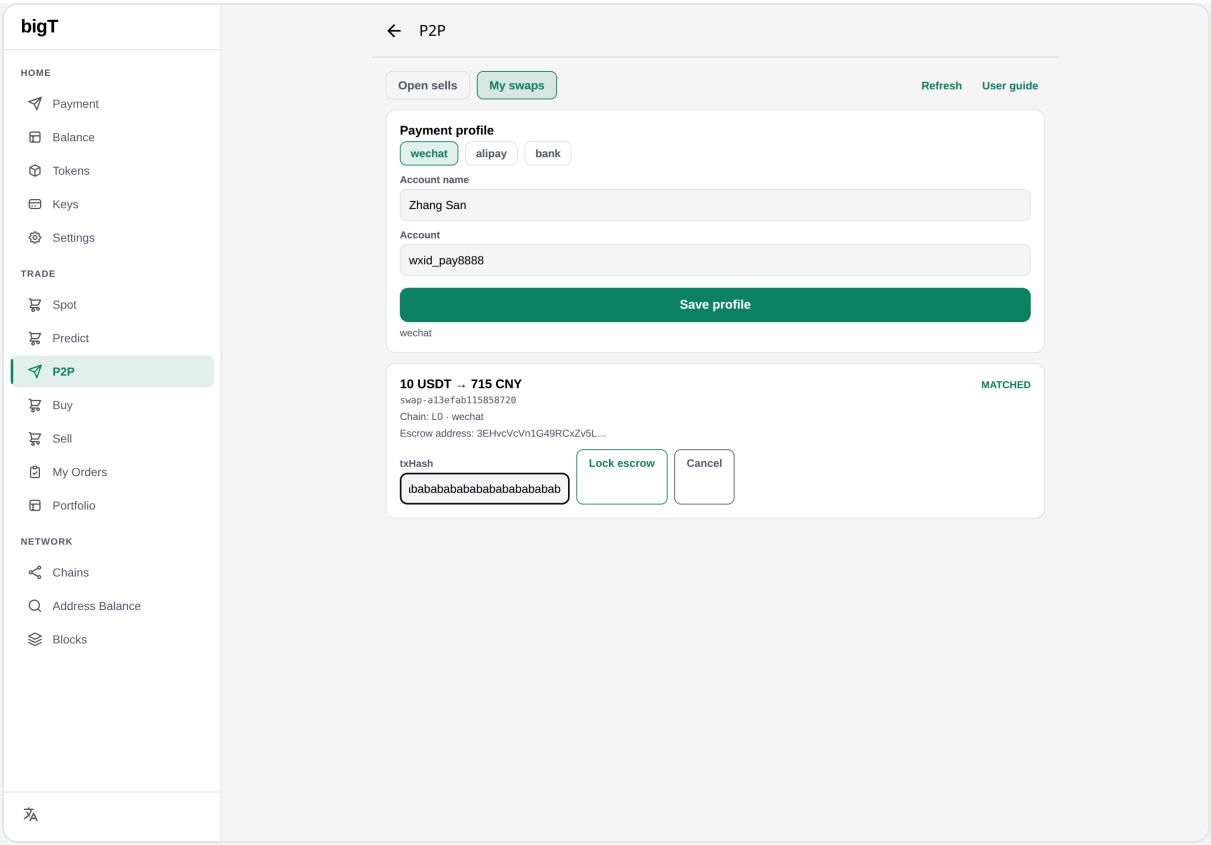
引擎为该生命周期创建一个唯一的 `swapId`，交易进入 `MATCHED`，显示在买方的 **My swaps（我的交易）** 标签页：



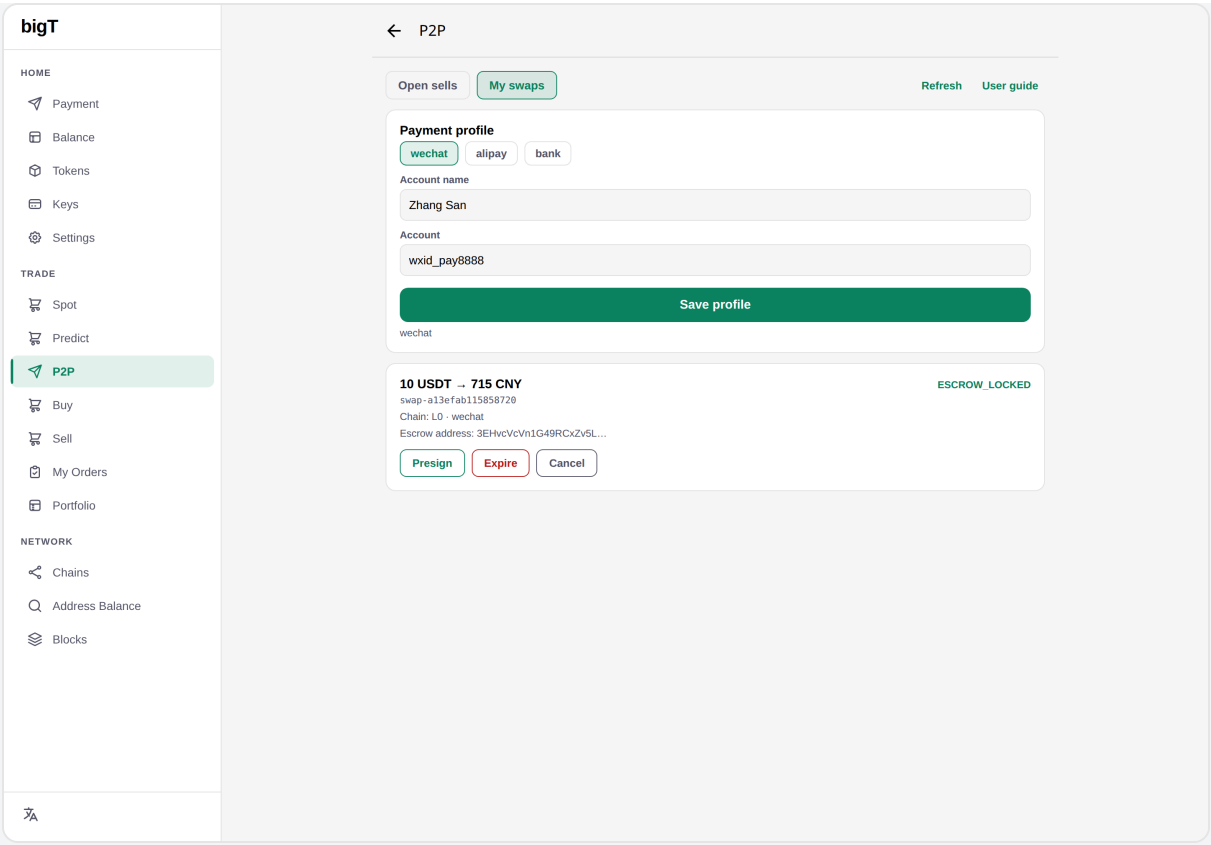
---

4. 托管锁定 — 卖方为 L0 注资

卖方把代币发送到确定性的 2-of-3 托管地址，并上报该笔转账的 txHash。引擎从链上证明锁定（CONFIRMED、目标为 escrowAddress、金额匹配），任一检查不通过则**失败即关闭**：



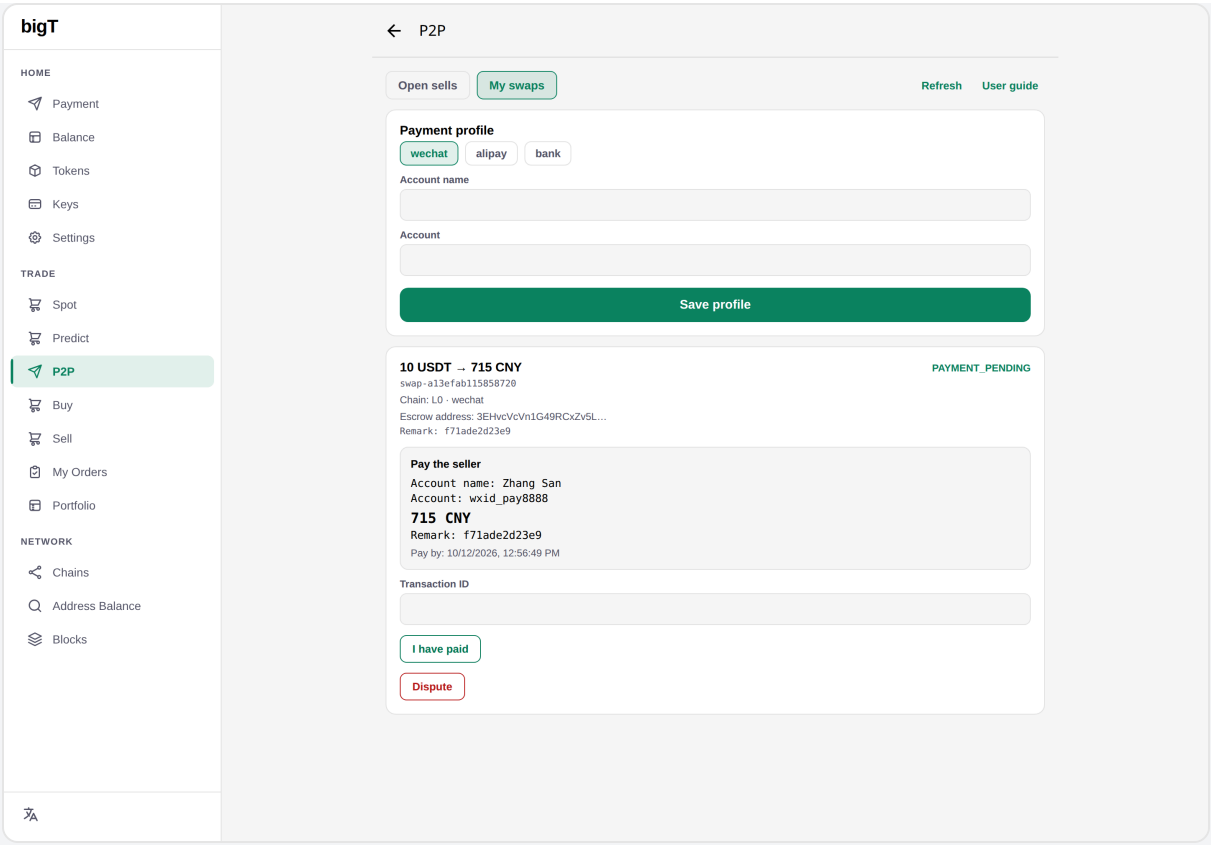
交易此时为 `ESCROW_LOCKED` 。买方在支付任何法币**之前**就能看到资金已被锁定，并获得一个 **`Payment details`（付款详情）**操作：



---

5. 买方拉取付款说明

买方点击 **`Payment details`（付款详情）**；引擎会披露卖方的收款资料，并生成一个转账*必须*附带的**每笔交易备注码**，以及精确金额和截止时间。说明会内联渲染在交易卡片中：

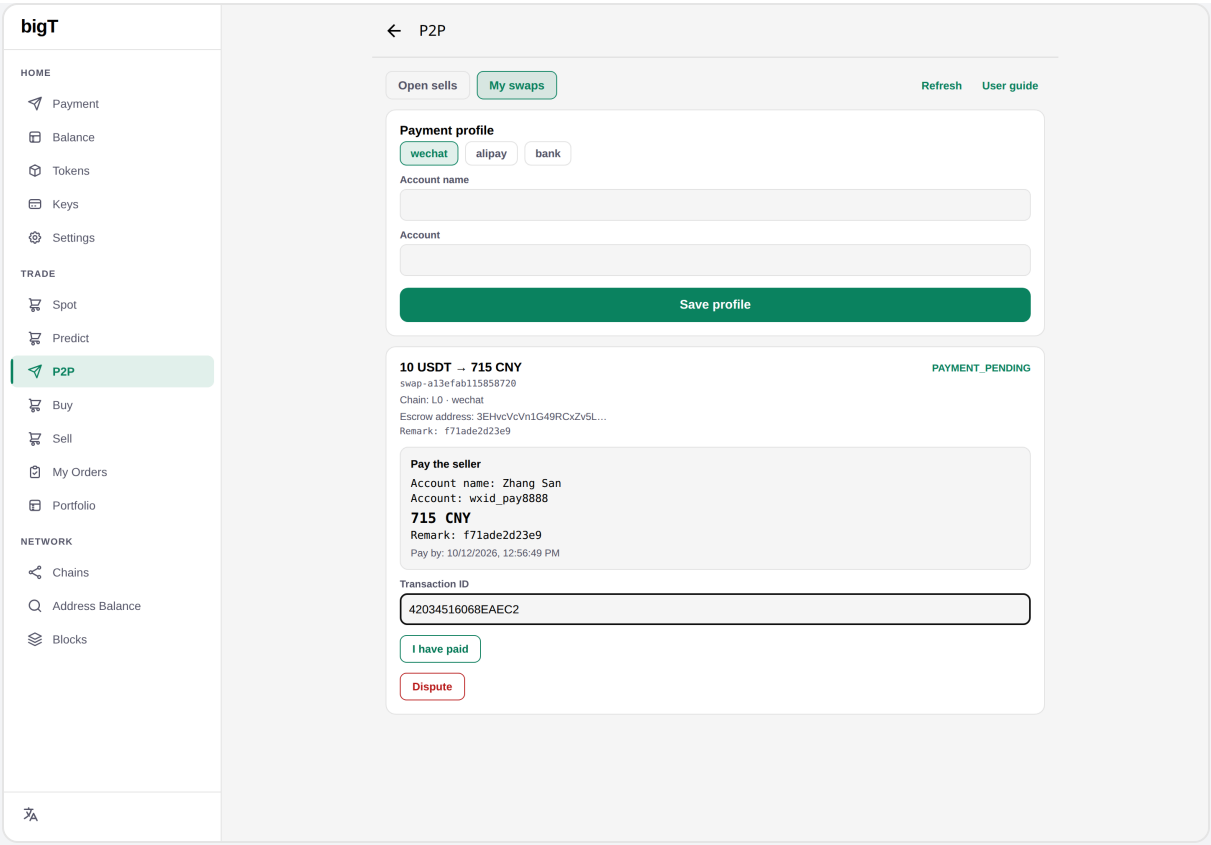


交易为 `PAYMENT_PENDING`，并启动一个超时计时器——若买方始终未付款，卖方无需买方同意即可让其过期并退还托管。

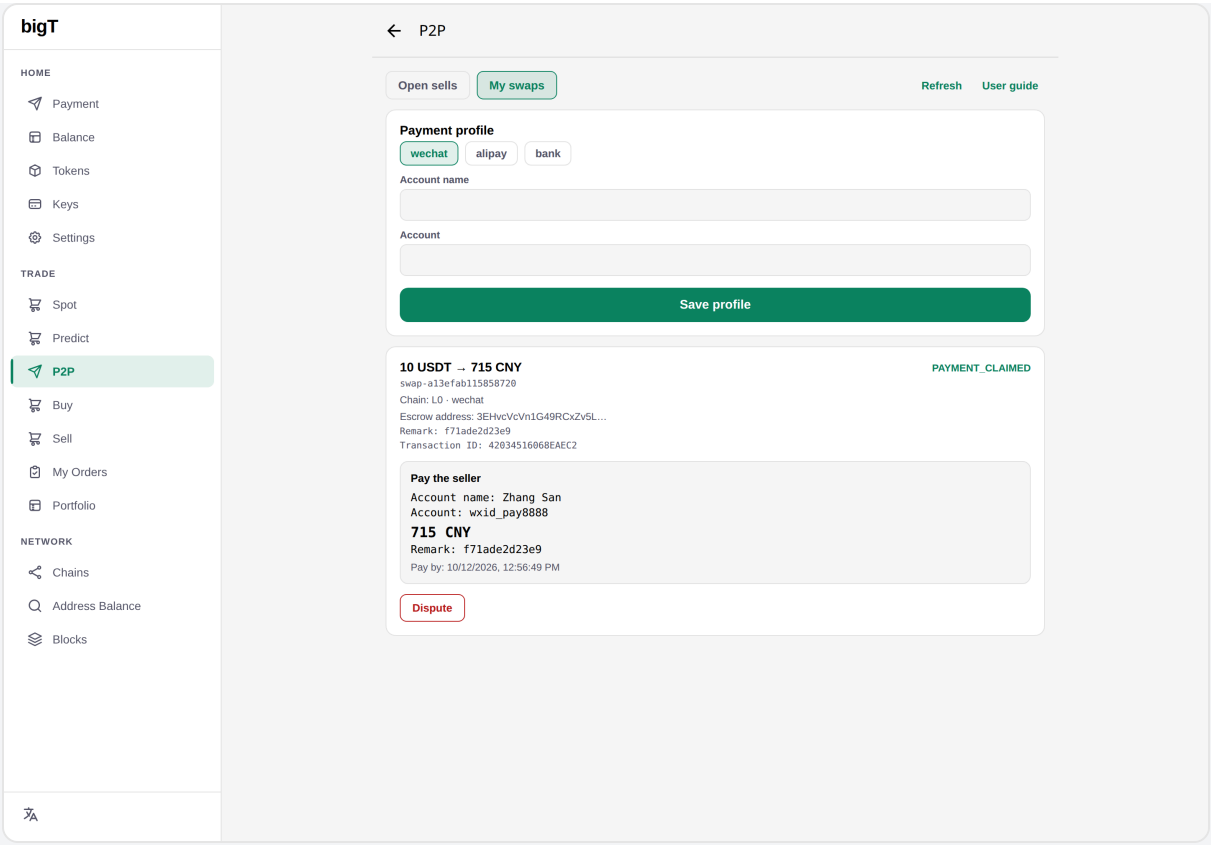
---

6. 买方付款并用流水号认领

买方在自己的微信/支付宝/银行应用里转账精确金额，将交易号（流水号）粘贴到 **`Transaction ID（交易号）`**，然后点击 **`I have paid（我已付款）`**。可选的收据图片存放在服务端；只有它的 `sha256` 会被锚定到交易事件上：



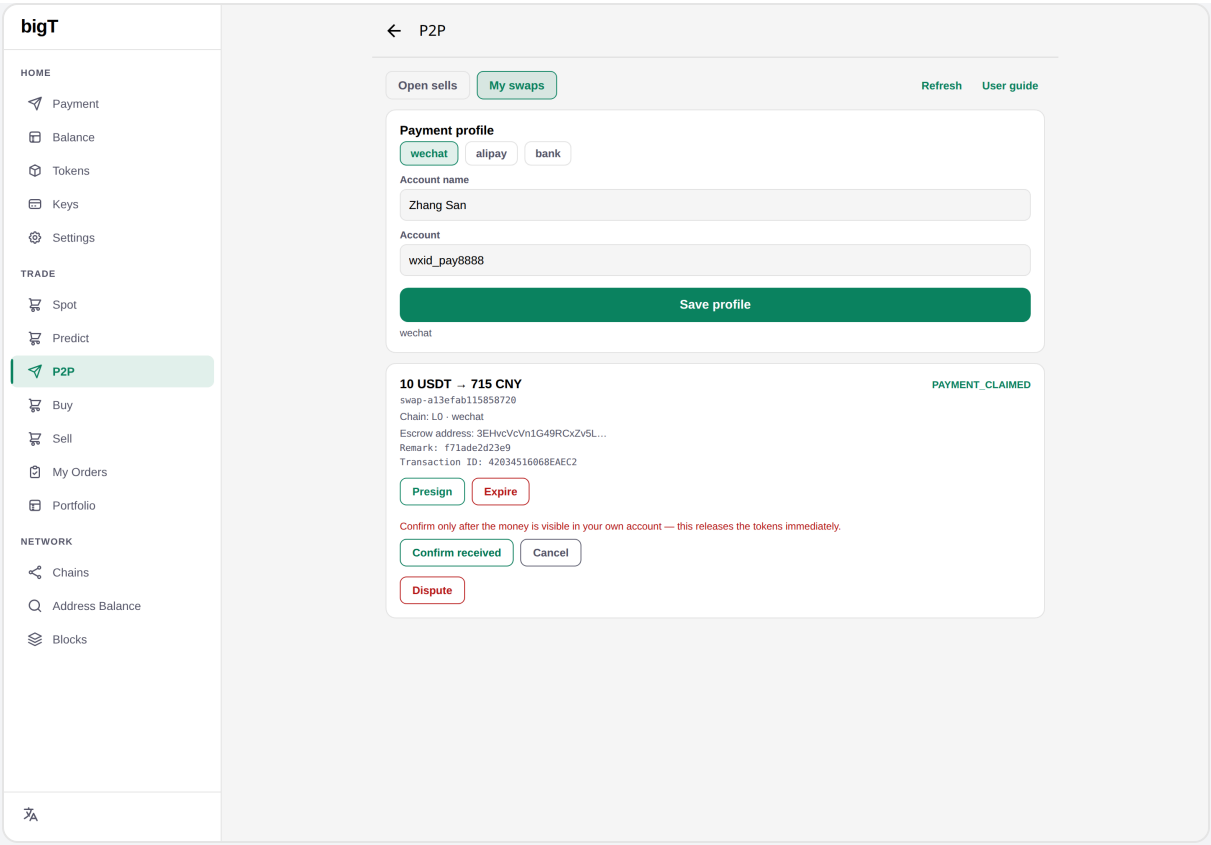
状态现在是 `PAYMENT_CLAIMED` —**代币仍然被锁定。** 这个状态纯粹代表买方的声明：



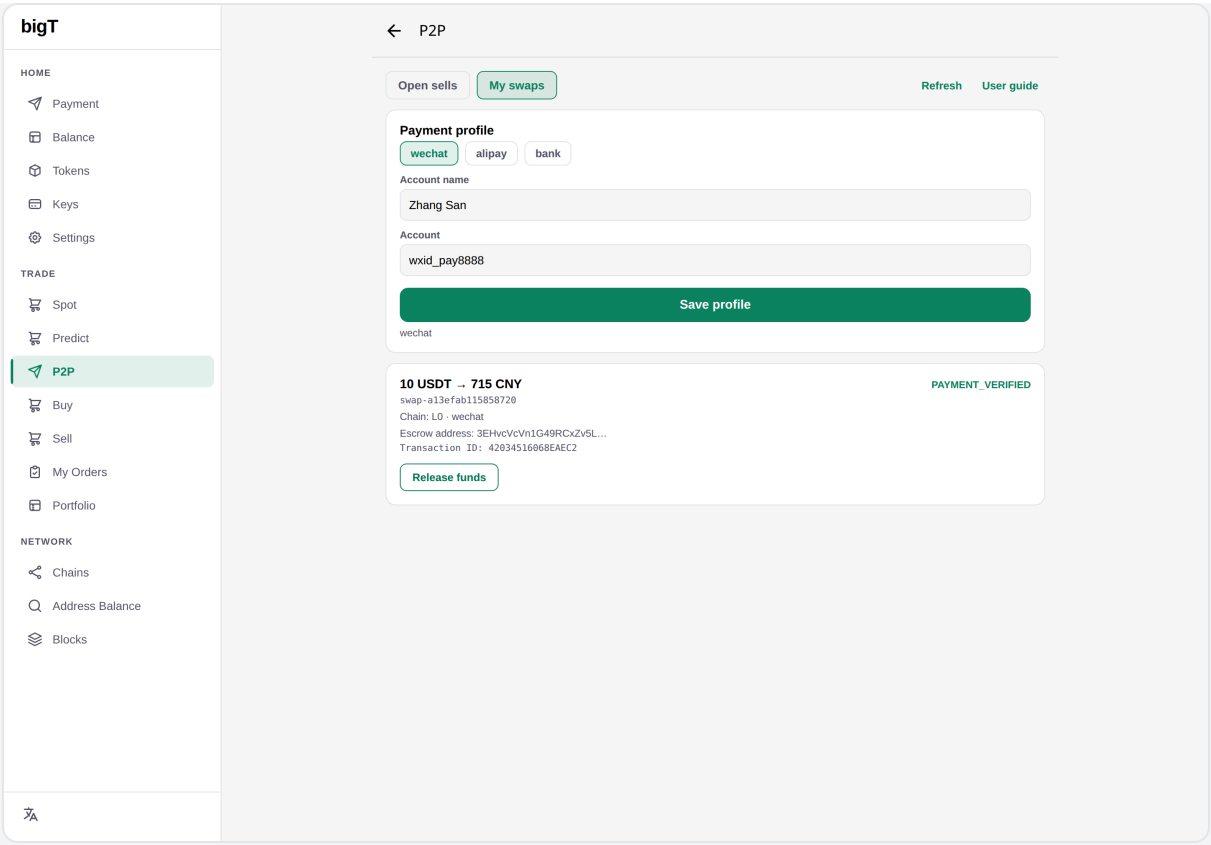
---

7. 卖方在自己的账单上确认

卖方的卡片会显示买方的流水号和备注。卖方在**自己的账户里核对金额 + 备注 + 时间 + 流水号**，然后才点击 **Confirm received（确认已收款）**：

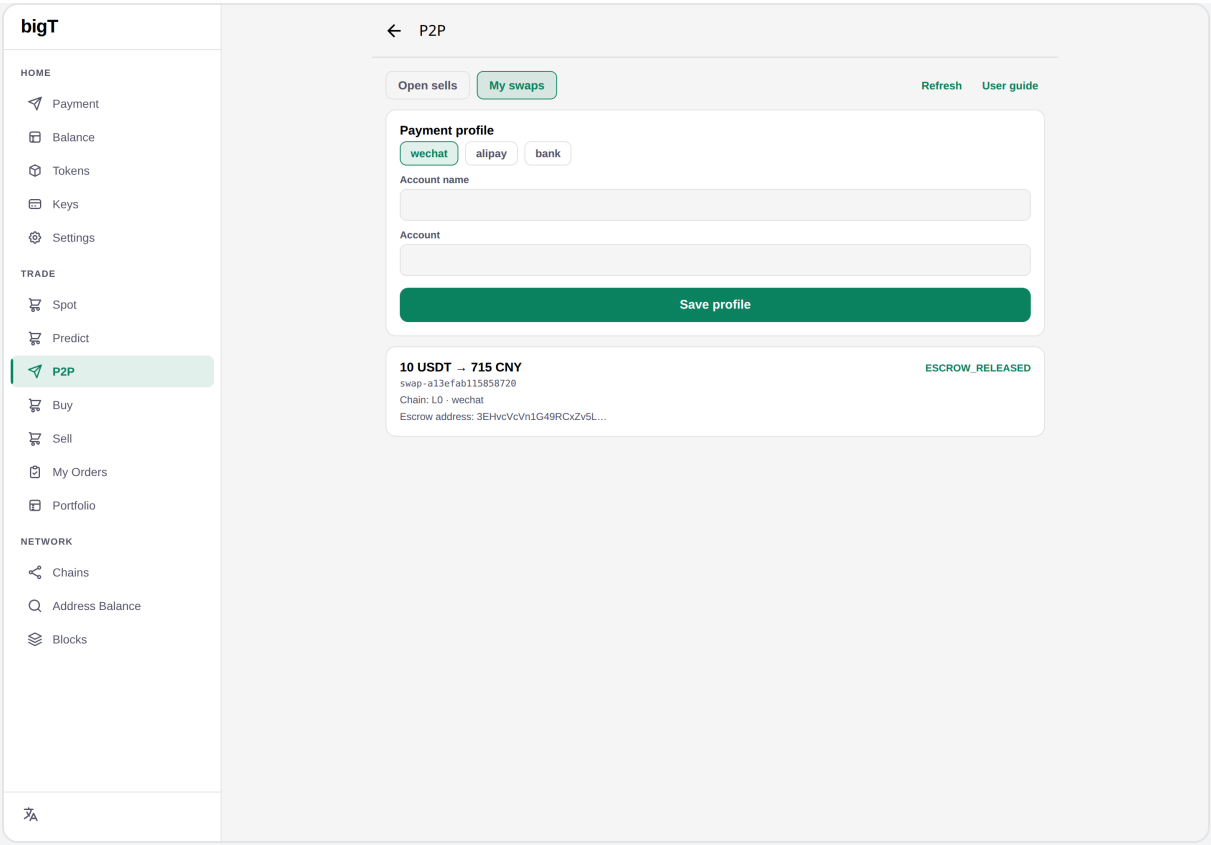


交易为 PAYMENT_VERIFIED：

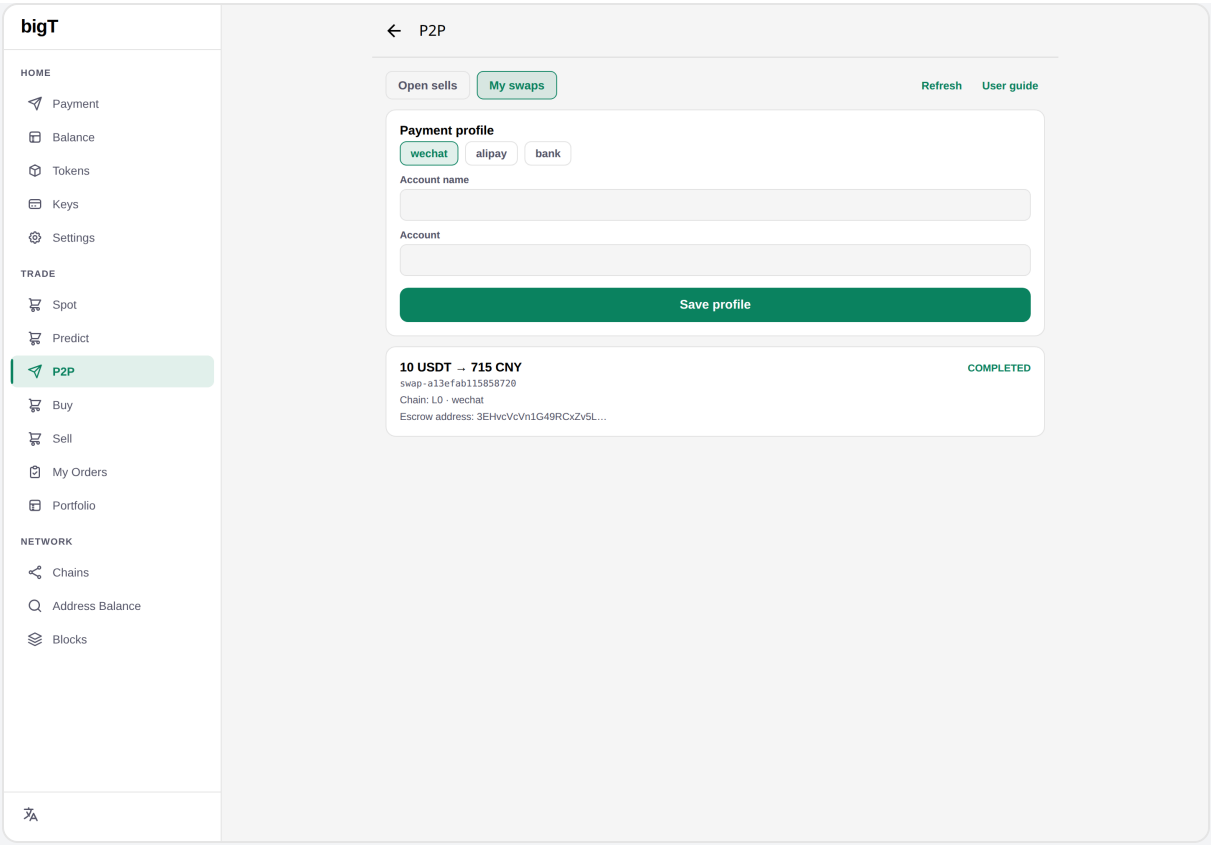


8. 释放 — 资金转移

释放会移动托管的代币：卖方（预签名或共同签名）与引擎各自对托管输出的支出签名，两个签名即可满足 `2-of-3` 脚本。代币到达买方的收款地址：



由于卖方是直接收款的，因此没有法币出金步骤—交易就此完成：



什么在保护你

风险	缓解措施
对手方走人	资金存放在 2-of-3 P2SH 地址中；任何一方都无法单独移动
少付或多付	金额在撮合时即固定；说明会带上精确数字和每笔交易的备注
伪造付款声明	仅凭买方的声明永远不会释放——卖方要在自己的账单上确认
卖方声称未收到款	仲裁；链上记录和证明存储即为证据
引擎作恶	仅引擎可执行的步骤需要引擎 DID 签名；每次转换都被锚定且可公开审计
争议	任一方都可冻结交易；代币保持锁定直至争议解决
退款	`expire → refund`，由引擎与卖方共同签名——无需买方同意，无时间锁

上线运行

演示运行在仓库内、使用内存存储且不做链上检查的结算服务上。真实流程会把托管签名器接入 `L0` 广播步骤，并让引擎对接 `L1-SOCIAL` 来承载订单簿和审计锚定；买方直接付款给卖方，引擎仍然只能共同签名。